

29-01-26 Programs:

1. (a) Given two non-negative integers, num1 and num2 represented as string, return the sum of num1 and num2 as a string. You must solve the problem without using any built-in library for handling large integers such as BigInteger in Java or Python's arbitrary-length int. You must also not convert the inputs to integers directly. Implement a Python or Java method to solve it.
(b) Analyse the Time complexity of your algorithm.
2. (a) Define Modular Binary Exponentiation and express it as a recurrence relation.
(b) Implement both recursive and iterative versions of a Python or Java method to compute (aⁿ) mod m using Modular Binary Exponentiation.
3. (a) Explain why direct multiplication, (a*b) modulo m can overflow and define the recurrence relation of fast modular multiplication.
(b) Implement both recursive and iterative methods in Python or Java to compute (a · b) mod m. Analyse the time and space complexity.
4. Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if inserted in order. You must write an algorithm (Java/Python) with O(logn) runtime complexity.
Example: nums = [1,3,5,6], target = 5 → Output = 2.
Example: nums = [1,3,5,6], target = 7 → Output = 4

5. The pseudo code of the Bubble sort algorithm is given below

```
procedure BUBBLE_SORT(array A)
    n ← length(A)

    for i ← 0 to n - 1 do
        for j ← 0 to n - i - 2 do
            if A[j] > A[j + 1] then
                swap A[j] and A[j + 1]
            end if
        end for [End Inner Loop]
    end for [End Outer Loop]
end procedure
```

- a) Determine the Best case and worst-case Time complexity in Big-O Notation of this algorithm
- b) Optimize the algorithm to stop early when the array is already sorted, and implement the improved version in Python or Java.[3 Mark]

6. In a college, the exam department has a pile of student answer sheet IDs placed in random order. To prepare them for digital scanning, the department wants to arrange the IDs in ascending order.

They decide to use the following method:

- Repeatedly compare two adjacent IDs.
- If the first ID is larger than the second, swap them.
- Continue this process until the IDs are fully sorted.
- If during a complete pass no swaps are made, it means the IDs are already sorted and the process should stop early.

(a) Write a program in Python or Java to implement this sorting method.

(b) Apply the early-stopping optimization if the answer sheets are already sorted.

(c) State the time and space complexity with and without optimization.